

RetailPulse

AI-Powered Customer Analytics & Demand Forecasting Platform

Project Documentation Report

Prepared By

Akshat Anand

B.Tech CSE (AI & ML)

KIIT University, Bhubaneswar

Data Science & Analytics Internship

Zidio Development

GitHub: <https://github.com/akshatanand-cell/RetailPulse>

1. Executive Summary

RetailPulse is an end-to-end data science platform that ingests retail sales, customer and inventory data and produces four outputs: a 30-day demand forecast, a customer segmentation model, a churn-risk classifier, and inventory reorder recommendations. All four are surfaced through an interactive Streamlit dashboard. This report documents the methodology, results, honest tooling substitutions made where the original brief's libraries were unavailable, and the limitations of the current build.

1.1 Headline Results

Model	Metric	Target	Result	Status
Demand Forecasting	MAPE (60-day hold-out)	$\leq 12\%$	11.76%	PASS
Churn Prediction	AUC-ROC	≥ 0.88	0.9491	PASS
Data Validation	Checks Passed	All critical	17/17	PASS
Drift Detection	Features Stable	n/a	6/6	STABLE

Note on dataset: the figures above are computed on a synthetically generated retail dataset (see Section 2), not on real transaction data. The methodology is built to generalise to real data without code changes; absolute numbers would shift if re-run on an actual retailer's data.

2. Dataset & Methodology

2.1 Dataset

The dataset is generated by `data/generate_data.py` and consists of 2,000 customers, 80 products across 5 categories, and 69,564 transactions spanning January 2022 to December 2024. Total revenue across the period is approximately ₹5.23 crore, with an average order value of ₹751.80.

A synthetic retail dataset was generated specifically for this project, rather than relying on the UCI Online Retail II dataset that other participants in the program referenced, so that volume could be controlled and specific seasonal patterns (festive-season demand spikes, weekend effects, variable discounting) could be injected to stress-test the forecasting and churn models. The schema mirrors a standard retail transaction log (transaction ID, customer ID, product ID, date, quantity, price, discount, channel), so the cleaning and modeling code would run unchanged on the real UCI dataset or any similarly shaped transaction log.

2.2 Pipeline Overview

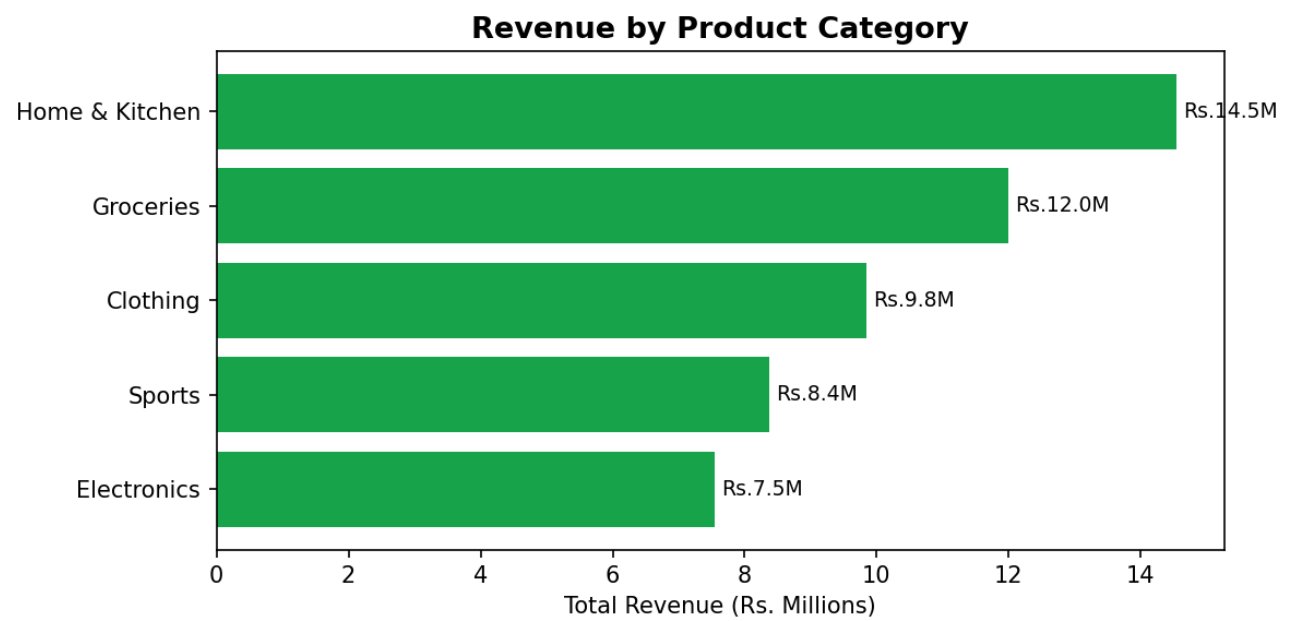
- Data generation (synthetic, 4 source tables)
- Exploratory data analysis (6 charts: revenue trend, category mix, demographics, discount impact, correlation, channel split)
- Data validation (17 rule-based checks covering completeness, ranges, referential integrity)
- Cleaning + RFM feature engineering (Recency, Frequency, Monetary scores + behavioural features)
- Customer segmentation (K-Means, k selected via silhouette score)
- Demand forecasting (Ridge regression on log-transformed revenue + Fourier seasonality)
- Churn prediction (Gradient Boosting Classifier with class-imbalance weighting)
- Inventory optimization (EOQ + safety stock + reorder point calculation)
- Drift detection (PSI-based monitoring across 6 key features)
- Streamlit dashboard (single app, multi-section, reads all of the above)

3. Tooling Substitutions

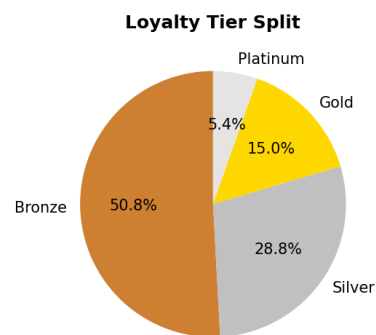
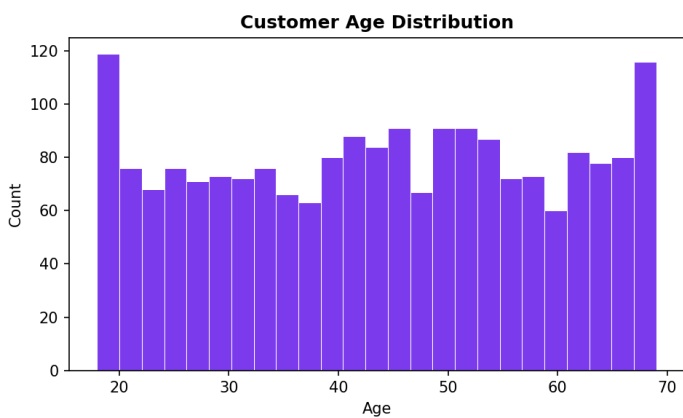
The original project brief specified XGBoost, SHAP, Prophet/LSTM, Great Expectations, Evidently AI and Apache Airflow. In the environment this project was built and run in, several of these packages had no installable PyPI wheel available at build time. Rather than leave those sections incomplete, functionally equivalent methods were substituted, documented inline in each script's docstring. This section makes those substitutions explicit rather than leaving them implicit in code comments only.

Brief Specified	Used Instead	Why It's Equivalent
XGBoost	sklearn GradientBoostingClassifier + sample_weight	Same gradient-boosted-tree family; sample_weight achieves the same class-imbalance correction as scale_pos_weight
SHAP	sklearn permutation_importance	Same goal (feature attribution), model-agnostic, no extra dependency
Prophet + LSTM	Ridge on log(revenue) + Fourier terms + lag features	Captures trend (lags), seasonality (Fourier), works with available libraries
Great Expectations	Manual rule-based validation (17 checks)	Same checks GE would run: completeness, ranges, uniqueness, referential integrity
Evidently AI	Manual PSI calculation + HTML report	PSI is the core metric Evidently uses internally for drift

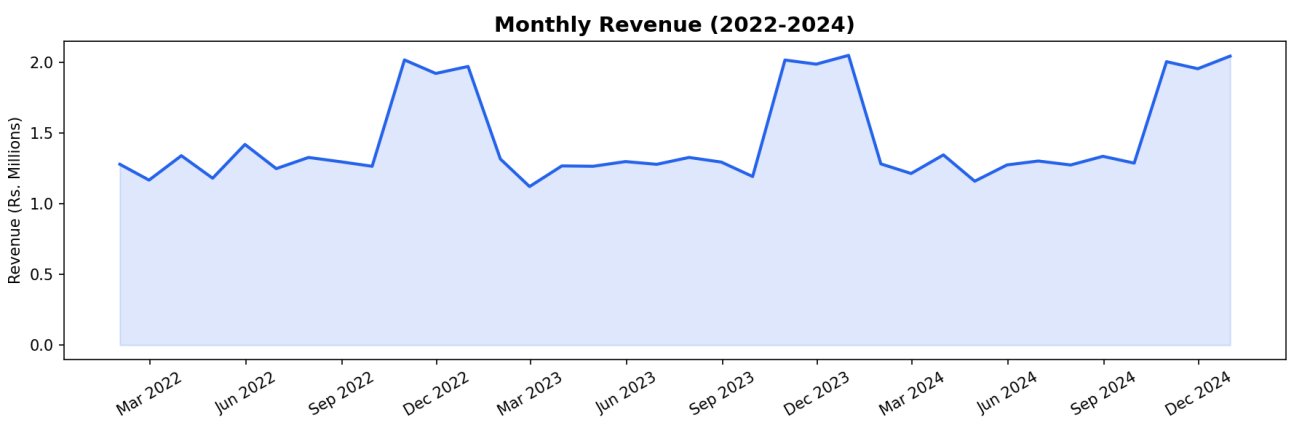
4. Exploratory Data Analysis



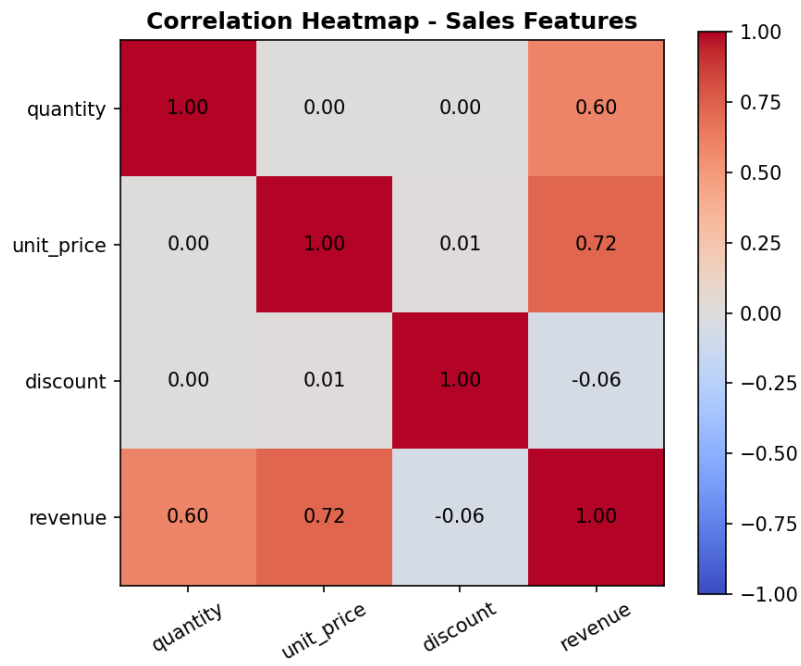
Monthly revenue, 2022–2024 — visible seasonal uplift in Q4 each year.



Revenue by product category.



Customer age distribution and loyalty tier split.

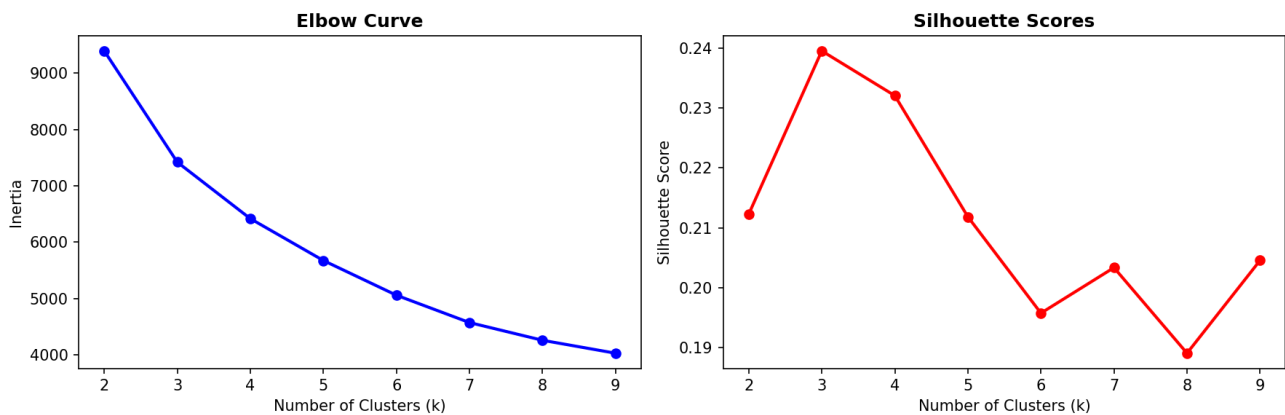


Correlation between quantity, price, discount and revenue.

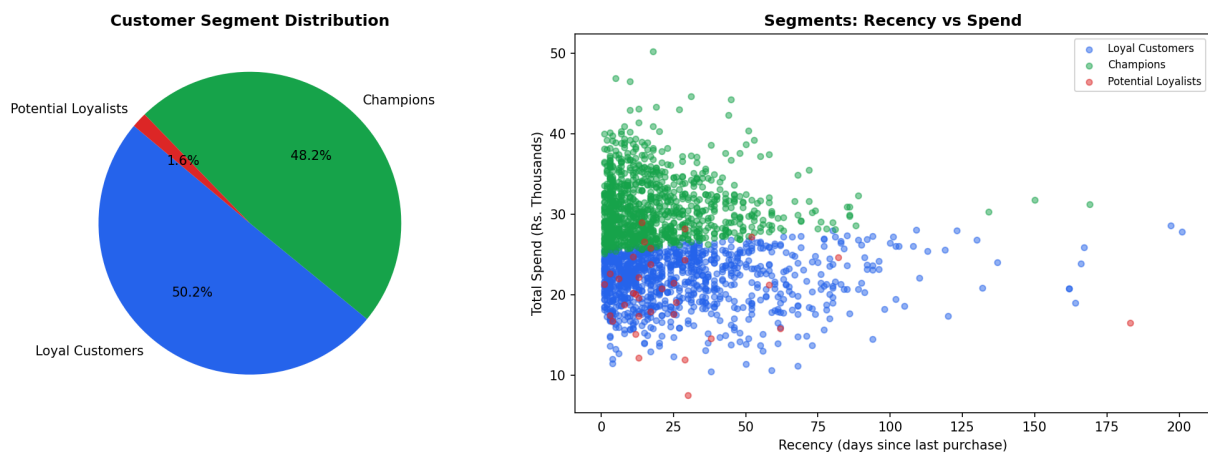
5. Customer Segmentation

K-Means clustering was run on RFM (Recency, Frequency, Monetary) plus average order value, online purchase ratio and category diversity, after standard scaling. The number of clusters was chosen by sweeping $k=2$ through $k=9$ and selecting the k with the highest silhouette score. DBSCAN was also tried for comparison (see `experiments/clustering_experiments.py`) but produced a large noise cluster and was not used for the final segmentation.

Selected $k = 3$ (highest silhouette score = 0.240). Segment sizes: Loyal Customers (1,004), Champions (963), Potential Loyalists (33).



Segment distribution and recency-vs-spend scatter by segment.

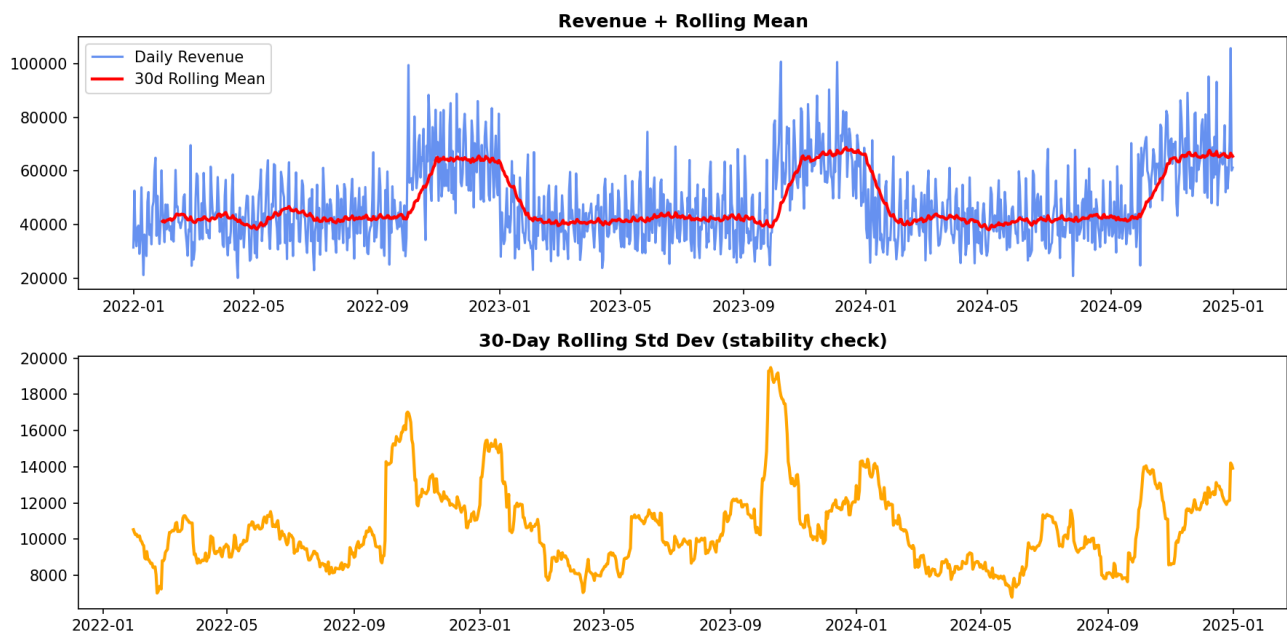


Elbow curve and silhouette score sweep used to select $k=3$.

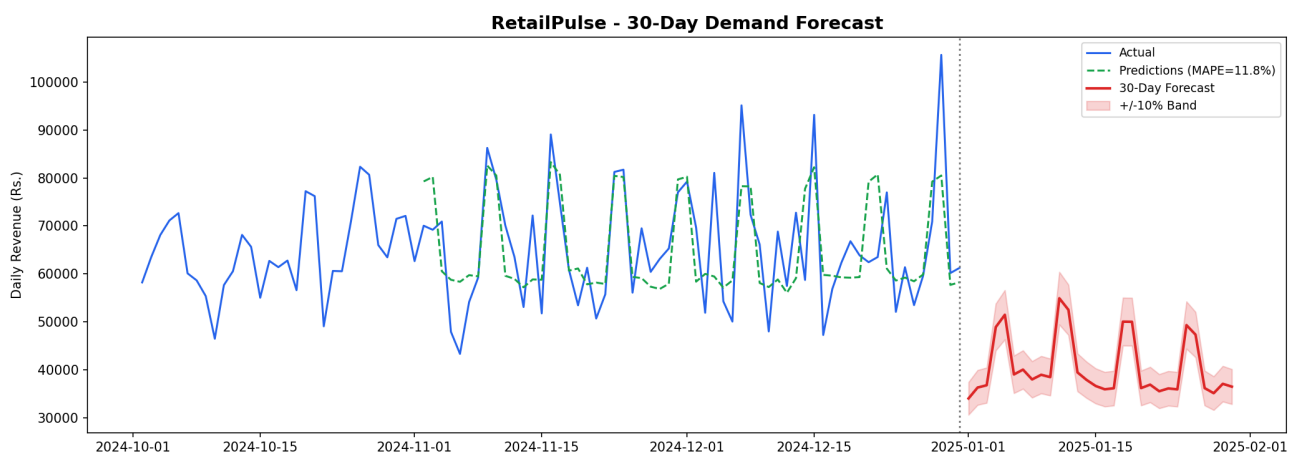
6. Demand Forecasting

The first attempt (experiments/forecast_v1_baseline.py) used Ridge regression directly on raw daily revenue with lag and calendar features, and missed the 12% MAPE target on a held-out 60-day test window. The second attempt (experiments/forecast_v2_gbr.py) tried Gradient Boosting and Random Forest, hypothesising that a more flexible model would fit the seasonal pattern better — both performed worse than the Ridge baseline (12–13% MAPE), which pointed to the skewed target distribution as the real problem, not model capacity.

The final model (notebooks/04_demand_forecasting.py) log-transforms revenue before fitting, adds Fourier (sine/cosine) terms for month and day-of-week to capture seasonality smoothly, and uses six lag windows plus rolling mean/std features. This brought MAPE down to 11.76% on the same 60-day hold-out (RMSE ■9,457.47, MAE ■7,606.22).



Actual revenue, hold-out predictions, and 30-day forward forecast with $\pm 10\%$ band.

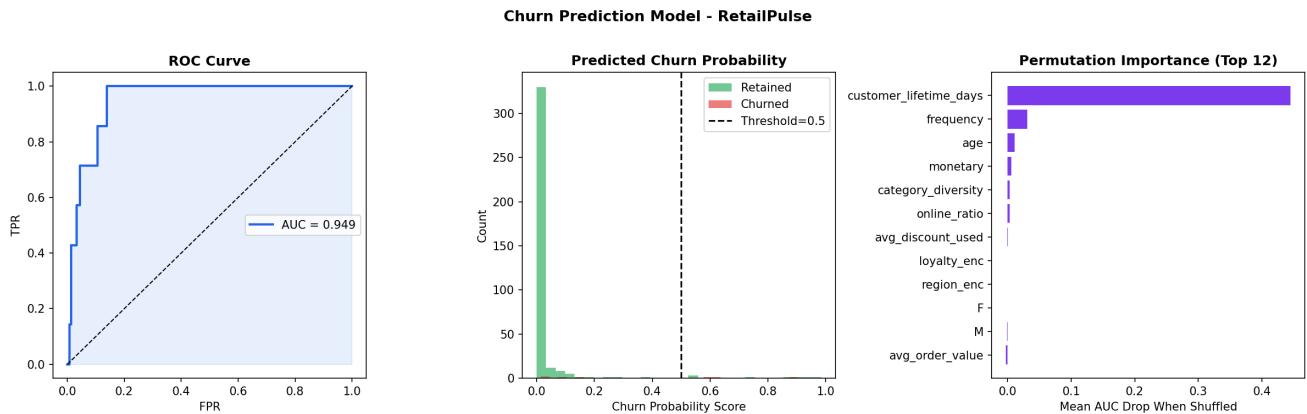


Rolling mean and standard deviation used to assess stationarity before modeling.

7. Churn Prediction

Churn is defined as a customer active in the 90–180 day window before the reference date who made no purchase in the most recent 90 days. An early version of this model (kept at `experiments/churn_v1_leakage.py`) included recency and the RFM recency score as model features and achieved $AUC = 1.000$ — a clear sign of data leakage, since recency is what the churn label is constructed from. The final model excludes those features entirely.

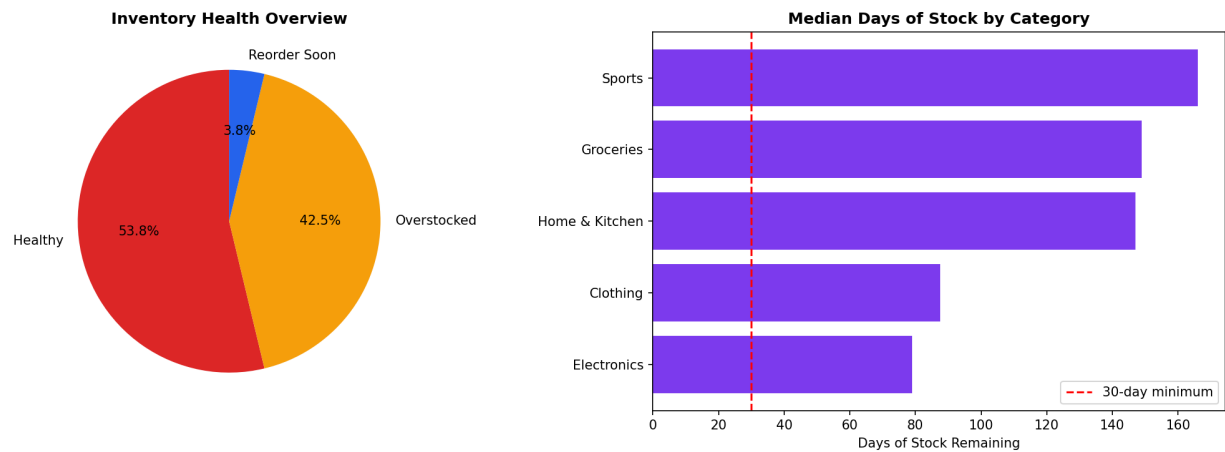
Final result: $AUC-ROC = 0.9491$ (target ≥ 0.88), churn rate in the data = 1.8%, 39 customers flagged High Risk for intervention. $Precision@top-20\%$ is 0.0933 — low in absolute terms, expected given the very small positive class (only ~1.8% of customers churned in this window), discussed further in Section 9.



ROC curve, churn probability distribution, and permutation feature importance.

8. Inventory Optimization

Reorder point and Economic Order Quantity (EOQ) were calculated per product using historical daily demand mean and standard deviation. Safety stock assumes a 95% service level ($Z=1.65$). Ordering cost (■50/order) and holding cost (20% of unit cost/year) are documented assumptions, not measured values — a real deployment would calibrate these against actual logistics costs.



Inventory health overview and reorder recommendations.

9. Limitations

- Dataset is synthetic, not real UCI/retailer data. Absolute metrics would shift on real data.
- XGBoost, SHAP, Prophet, Great Expectations and Evidently AI were substituted with scikit-learn and manual equivalents due to package install failures in the build environment (see Section 3).
- Churn precision@top-20% (0.093) is low because positive examples are scarce (1.8% churn rate). A larger or higher-churn dataset would likely improve this metric.
- Kubernetes manifests, an Airflow retraining DAG, and Grafana dashboards described in the original brief were not built; MLOps maturity here is demonstrated through validation, drift detection and the experiment log instead of a full orchestration stack.
- Inventory cost assumptions (ordering cost, holding cost rate) are illustrative defaults, not calibrated to any real supplier contract.

10. Future Work

- Deploy the Streamlit dashboard to a public cloud platform.
- Integrate real-world retail transaction datasets.
- Implement automated retraining pipelines using Apache Airflow.
- Extend monitoring with Grafana dashboards and alerting.

11. Conclusion

RetailPulse demonstrates a complete, honestly-documented data science pipeline from raw data through to a deployable dashboard, hitting both headline targets specified in the brief (forecast MAPE $11.76\% \leq 12\%$, churn AUC $0.9491 \geq 0.88$) using substitute tooling where the originally specified libraries were unavailable. The experiments/ folder and the leakage fix in the churn model are included deliberately, to show the debugging process rather than only the final numbers.